

Luffy AI

Network Evolution and Plasticity

9 March 2026

Project Goals

Non-stationary industrial environments are challenging for artificial neural agents to navigate.

- Standard control theory and fixed-parameter Deep Reinforcement Learning (DRL) methods don't adapt well to these changing environments over time.

Neuroevolutionary approaches, such as NEAT, offer a promising alternative to fixed-topology approaches, by allowing both the network structure and the hyperparameters to evolve, often yielding simple, efficient networks suitable for edge deployment.

By incorporating plasticity rules, such as Hebbian learning and Oja's rule, NEAT agents will be able to further adapt to changing environments, enabling resilient and autonomous industrial operations.

The project with Luffy.AI has three concrete goals:

- Extend NEAT with neuroplasticity rules.
- Benchmark against standard DRL methods.
- Run ablation studies - systematically remove rules to understand how each rule contributes to the performance and adaptability of NEAT.

Project Timeline

The plan has the following milestones (with completion dates):

1. Scope [definition](#) – 3 March
2. Literature review – 10 March
3. Control environment choice (Cartpole / walker etc) - 17 March
4. NEAT network creation – 24 March
5. Rule application – 1 April
6. Rule ablation study – 15 April
7. Output:
 - Flyer – 8 May
 - Impact assessment – 12 May
 - Video – 15 May

Initial Scope Decisions

1. Network size: NEAT will be initialised with the minimum number of nodes (just the input and output nodes), hidden nodes will only be added through evolution.
 - The impact that network size has on the effectiveness of the rules may be investigated later.
2. Environments: Initially, 2D environments will be examined. From Gymnasium; CartPole and LunarLander.
 - The 3d environments are more interesting, particularly halfcheetah/ant/humanoid, but are more complex and will be investigated at a later date.
3. Non-Stationarity: Initially a Proof of Concept (PoC) will be established using the simplest environmental factors.
 - For example, for CartPole, wind perturbations will be added – a non-linear, inconstant horizontal force on the pole.

Winning Neural Network — CartPole-v1



NEAT Winner · 6 nodes · 8 active connections · fitness 500

Neuroplasticity Rules

$$\text{fixed}(s, \alpha, \beta) = 0$$

$$\text{Hebb}(s, \alpha, \beta) = \alpha\beta$$

$$\text{Oja}_+(s, \alpha, \beta) = \beta(\alpha - s\beta)$$

$$\text{Oja}_-(s, \alpha, \beta) = \text{Oja}_+(s, \alpha, 1 - \beta)$$

$$\text{correlation}(s, \alpha, \beta) = \alpha(2\beta - 1)$$

$$\text{anticorrelation}(s, \alpha, \beta) = -\text{correlation}(\alpha, \beta)$$

where α is pre-synaptic activation, β is post-synaptic activation, and s is current synapse strength.

Update is computed as

$$\Delta s^t = \eta \text{rule}(s^t, \alpha, \beta)$$

$$s^{t+1} = s^t + \Delta s^t$$

where η is the learning rate. η , initial strength s^0 , and choice of rule are evolved properties of each *synapse*.

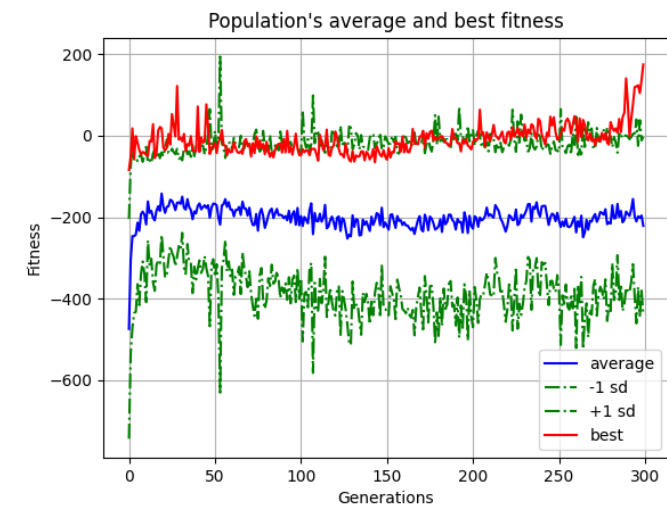
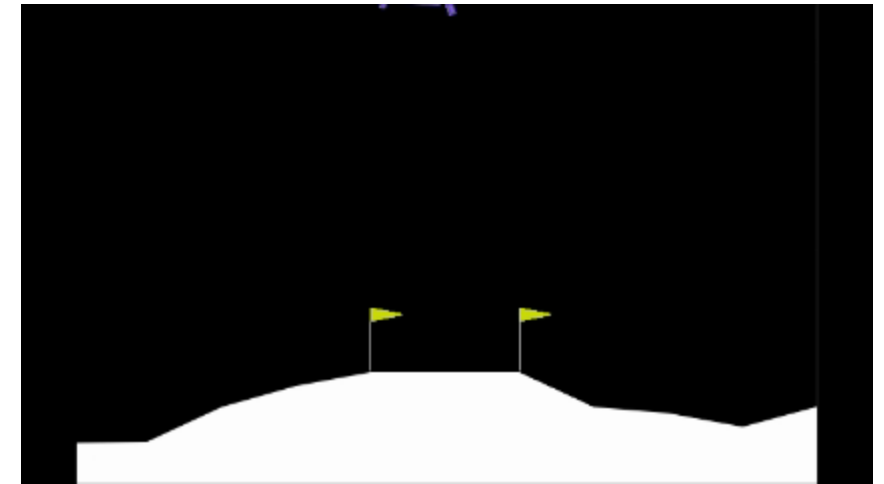
Options for Implementation

1. We produce a network topology using NEAT, and then test one plasticity rule at a time, at a global level, meaning that every connection between nodes in the network is updated using this universal rule. We vary the control environment (i.e. via wind or other environment shifts) and observe and analyse how the fitness is affected by each of the rules.
2. Same as option 1, except we perform a test that allows for each connection to use a different, randomly assigned, plasticity rule.
3. We modify the "simple" NEAT algorithm to allow the selection of plasticity rules as well as weights / connections during the mutation process. Different genomes would have different rules. Ablation would remove one rule at a time from the decision space and analyse the effect of doing so.

Initial Implementation

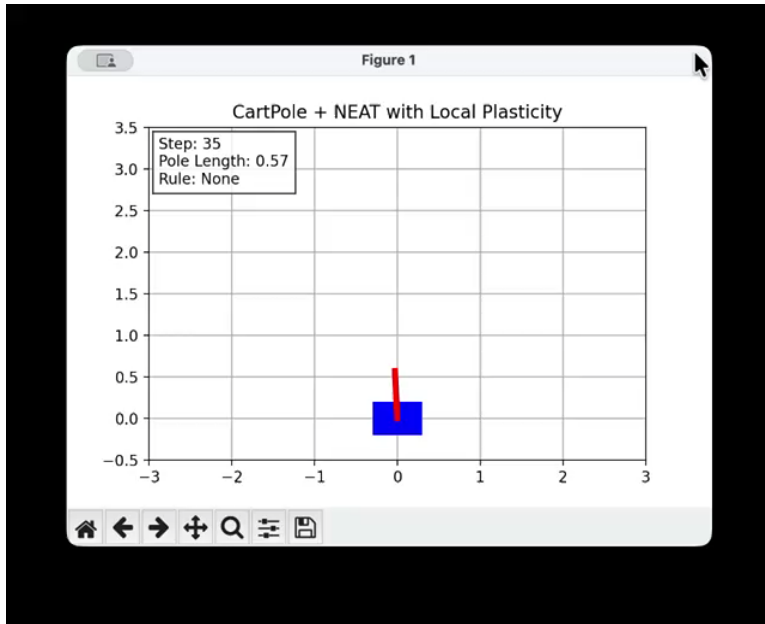
Initial implementation of NEAT in simulation environment used a lunar-lander control problem:

- ❖ Explored simulations of various difficulties (greater number of inputs, more complex outputs).
- ❖ No Hebbian (Oja etc.) learning at this stage.

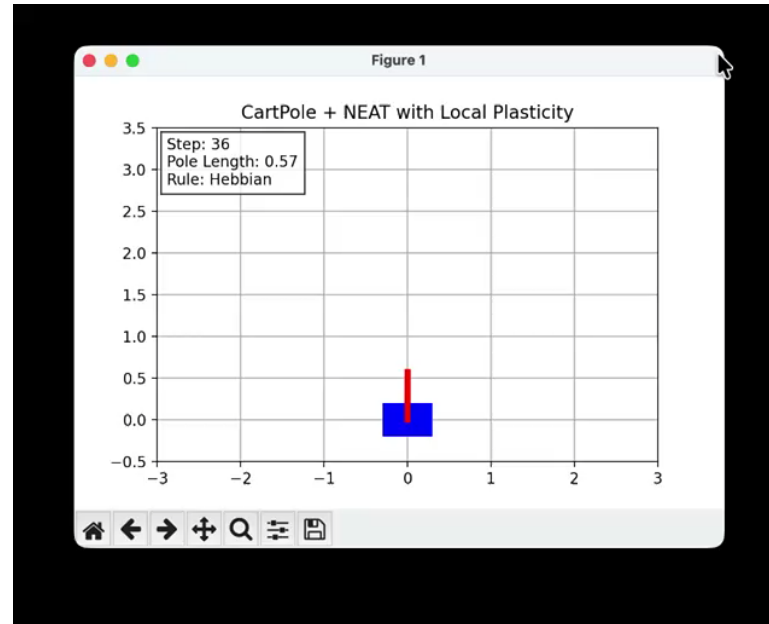


This Week's Progress

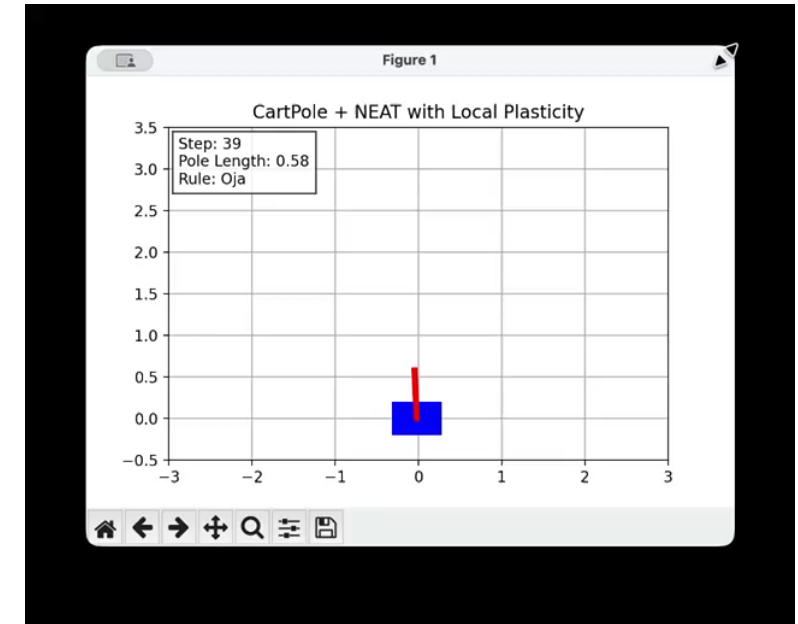
- No pre-existing libraries for NEAT and Hebbian/Oja plasticity rules.
- Completed Cartpole implementation using raw Python (no libraries). Changing environment is simulated by growing pole length.



No plasticity
Weights remain constant



Hebbian plasticity
Weights vary without limit



Oja plasticity
Weights evolve, but normalised

This Week's Progress

- No pre-existing libraries for NEAT and Hebbian/Oja plasticity rules.
- Completed Cartpole implementation using raw Python (no libraries). Changing environment is simulated by growing pole length.

Evolving for 20 generations using NONE plasticity...

Gen 01	Fitness: 212	Nodes: 6	Connections: 5
Gen 02	Fitness: 212	Nodes: 6	Connections: 5
Gen 03	Fitness: 212	Nodes: 6	Connections: 5
Gen 04	Fitness: 213	Nodes: 7	Connections: 7
Gen 05	Fitness: 404	Nodes: 6	Connections: 5
Gen 06	Fitness: 536	Nodes: 6	Connections: 5
Gen 07	Fitness: 538	Nodes: 6	Connections: 5
Gen 08	Fitness: 538	Nodes: 6	Connections: 5
Gen 09	Fitness: 779	Nodes: 7	Connections: 7

Solved!

Connection from: 0 to: 5 weight: 0.10
Connection from: 1 to: 5 weight: 0.27
Connection from: 2 to: 5 weight: 1.26
Connection from: 3 to: 5 weight: 1.40
Connection from: 4 to: 5 weight: 0.01
Connection from: 1 to: 6 weight: 1.00
Connection from: 6 to: 5 weight: 0.43

Evolving for 20 generations using HEBBIAN plasticity...

Gen 01	Fitness: 153	Nodes: 6	Connections: 5
Gen 02	Fitness: 153	Nodes: 6	Connections: 5
Gen 03	Fitness: 169	Nodes: 6	Connections: 5
Gen 04	Fitness: 199	Nodes: 6	Connections: 5
Gen 05	Fitness: 236	Nodes: 6	Connections: 5
Gen 06	Fitness: 285	Nodes: 6	Connections: 5
Gen 07	Fitness: 317	Nodes: 6	Connections: 5
Gen 08	Fitness: 345	Nodes: 6	Connections: 5
Gen 09	Fitness: 697	Nodes: 6	Connections: 5
Gen 10	Fitness: 697	Nodes: 6	Connections: 5
Gen 11	Fitness: 761	Nodes: 6	Connections: 5

Solved!

Connection from: 0 to: 5 weight: 0.06
Connection from: 1 to: 5 weight: -0.01
Connection from: 2 to: 5 weight: 1.43
Connection from: 3 to: 5 weight: 0.62
Connection from: 4 to: 5 weight: -0.01

Evolving for 20 generations using OJA plasticity...

Gen 01	Fitness: 64	Nodes: 6	Connections: 5
Gen 02	Fitness: 64	Nodes: 6	Connections: 5
Gen 03	Fitness: 115	Nodes: 6	Connections: 5
Gen 04	Fitness: 137	Nodes: 7	Connections: 7
Gen 05	Fitness: 149	Nodes: 8	Connections: 9
Gen 06	Fitness: 391	Nodes: 7	Connections: 7
Gen 07	Fitness: 800	Nodes: 7	Connections: 7

Solved!

Connection from: 0 to: 5 weight: 0.09
Connection from: 1 to: 5 weight: 0.02
Connection from: 2 to: 5 weight: 1.08
Connection from: 3 to: 5 weight: 0.31
Connection from: 4 to: 5 weight: -0.19
Connection from: 4 to: 6 weight: 1.00
Connection from: 6 to: 5 weight: 0.05

No plasticity

Weights remain constant

Hebbian plasticity

Weights vary without limit

Oja plasticity

Weights evolve, but normalised

NEAT + Learning Algorithm

- Initialise population with new genomes (each with 5 input, 1 output nodes)
- Reset best genome, best fitness tracker
- Loop for 20 generations:
 - Loop through each genome of the population:
 - Iterate cart until failure (angle, position > tolerance) or success
 - Calculate step dynamics (velocity, position change)
 - Pass state into neural network, generate action (push left / right)
 - Adjust weights using plasticity parameter (Hebbian, Oja etc.)
 - Increment fitness
 - Seed next generation with top 2 genomes (elitism)
 - Fill out remainder of generation with mutations (80% mutated weight, 10% mutated node, 10% mutated connection)
- Plot behaviour of best genome across all generations

Next Steps

- Implement the neuroplasticity rules in Python using a fixed network.
- Build a Hybrid NEAT-Neuroplasticity rule framework.
- Test the performance of this framework on more complex, non-stationary environments.
- Develop a statistical framework to smooth stochastic nature of individual simulated runs.